```
class Mixin
  constructor: (methods) ->
    for name, body of methods
      @[name] = body
  include: (klass) ->
    for key, value of @
      klass::[key] = value

htmlRenderer = new Mixin
  render: -> "rendered"

class Camera
  htmlRenderer.include @

leica = new Camera()

leica.render()
#rendered
```

The include method copies the methods of the current object to the supplied class.

Create a mixin.

Include the mixin into the Camera class.

A Camera instance has a render method added by the mixin.

By comprehending the properties of an existing object and setting them as properties on another object, you're able to mix in behavior from one object to another. Note that the mixin technique you've learned is static. Once the properties have been copied to the `Camera` class, further changes to the `htmlRenderer` object are not seen by `Camera` instances. Mixins created in this way allow you to do a one-time copy of related functions to another object. More sophisticated dynamic programming is demonstrated in later chapters.

5.7.3. Example: enumerable mixin

Remember the accumulator from chapter 3 that you used to sum an array of numbers? Accumulating numbers wasn't exciting, but it did demonstrate how to pass a function as an argument to another function:

```
accumulate = (initial, numbers, accumulator) ->
  total = initial or 0
  for number in numbers
    total = accumulator total, number
  total

sum = (acc, current) -> acc + current
accumulate 0, [5,5,5], sum
# 15
```

The accumulate function from chapter 3